DAT535 Project Report

Spark pipeline for TLC Yellow Taxi data

Vegar Kleiva Hauge
University of Stavanger, Norway
272036@uis.no

Fredrik Nilsen Låder
University of Stavanger, Norway
267204@uis.no

ABSTRACT

This report presents a distributed Spark pipeline for 79,500,000 NYC Yellow Taxi trip records, implemented as a three-layer Medallion Architecture.

The Bronze layer ingested 24 monthly Parquet files totaling 1.24 GB while enriching every record with comprehensive metadata and flagging 10.35% of records for quality issues without any data loss. The Silver layer performed pure RDD-only cleaning and enrichment, removing 3.15% of critically invalid records and adding nine human-readable descriptive fields to the remaining 76.97 million rows. The Gold layer produced seven analytical datasets optimized for revenue analysis, zone performance metrics, and temporal pattern detection.

A systematic cluster experiment compared four configurations sharing the same 16-core, 27 GB hardware pool against a single-VM baseline with 4 cores and 8 GB RAM, that took 55 minutes. The fastest full-allocation setup used seven smaller workers and finished in 24.2 minutes. A larger node four-worker configuration achieved nearly identical performance using 43% fewer nodes. Most notably, deliberately constraining the same hardware to only 8 cores and 44% memory increased runtime by just 33% , using 32.9 minutes while improving per-core efficiency by 50% .

These results reveal pronounced diminishing returns beyond eight cores for this CPU-bound workload and demonstrate that constrained resource allocation yields superior cost-efficiency in non-time-critical batch jobs through reduced coordination overhead. The findings provide actionable guidance for production Spark deployments and, together with the fully RDD-based implementation, offer a replicable framework for large-scale data engineering projects requiring fine-grained performance control.

KEYWORDS

Report writing, Latex tips

1 INTRODUCTION

Context and Motivation. The rapid growth of data-intensive applications has created an urgent need for a scalable data processing infrastructure capable of handling billions of records with acceptable latency and resource efficiency. Modern organizations increasingly rely on data lakes and distributed processing frameworks to transform raw operational data into actionable business intelligence. Apache Spark [3] has emerged as the de facto standard for large-scale data processing, offering fault-tolerant distributed computing through its Resilient Distributed Dataset (RDD) abstraction and higher-level DataFrame APIs. However, deploying Spark in production environments presents significant challenges: cluster resource allocation must balance processing performance against operational costs, workload characteristics heavily influence optimal configurations, and naive scaling strategies often yield diminishing returns due to coordination overhead and network bottlenecks.

The NYC Taxi & Limousine Commission (TLC) publishes monthly travel records for all licensed taxi services [8], generating datasets containing tens of millions of records with complex schemas that require data quality validation, schema normalization, and semantic enrichment before business analysis. Processing such datasets efficiently requires careful architectural design: raw data ingestion must preserve lineage and detect quality issues without filtering; cleaning operations must handle heterogeneous schemas and apply domain-specific validation rules using low-level transformations; and analytical outputs must be optimized for diverse business intelligence queries. Furthermore, understanding how resource allocation impacts both performance and cost-efficiency is critical for organizations operating under budget constraints or sharing computational resources across multiple workloads.

Research Problem. This work addresses the challenge of designing, implementing and optimizing a production-grade distributed data pipeline for processing 79.5 million NYC taxi trip records using Apache Spark. Specifically, we investigate three key problems: How can we implement a multi-layer data architecture using mainly RDD-level operations, without relying on high-level abstractions, ensuring fine-grained control over distributed computations? What data quality issues exist in real-world operational datasets, and how can we flag anomalies while preserving the complete data lineage for audit purposes? Most critically, how do different cluster resource allocation strategies affect per-core efficiency, and can constrained resource configurations achieve superior cost-performance ratios compared to full utilization?

The problem is non-trivial for several reasons. First, the dataset consists of field name variations and data type inconsistencies that prevent naive schema merging strategies. Second, the project constraint of using mainly RDD operations that requires manual implementation of operations typically handled by Spark's Catalyst optimizer. Third, cluster performance optimization is highly workload-dependent: CPU-bound operations, shuffle-heavy transformations, and memory-intensive aggregations respond differently to resource scaling. Understanding where diminishing returns occur requires systematic experimentation across multiple configurations, measuring not only absolute throughput but also per-core efficiency metrics that reveal coordination overhead invisible in wall-clock-time measurements.

Related Work. The Medallion Architecture pattern for Bronze, Silver and Gold layers has become an industry standard promoted by Databricks [6] and adopted widely for production data pipelines.

Data-intensive Systems and Engineering (DAT535), IDE, UiS
2025.

The architecture ensures data quality improvement through progressive refinement while maintaining complete lineage from raw sources to analytical outputs. Apache Spark's RDD abstraction [4] provides fault-tolerant distributed computing through lineage-based recovery, enabling large-scale data processing without traditional distributed storage replication overhead. MapReduce [7] established the programming model of mapping functions over partitioned data and reducing results through aggregation, which Spark extends with in-memory caching and richer transformations.

Performance optimization of Spark deployments has been studied extensively. Ousterhout et al. [9] analyzed bottlenecks in data analytics frameworks, demonstrating that task serialization, scheduler overhead, and straggler tasks significantly impact performance. However, their work focused on framework-level optimizations rather than resource allocation strategies for specific workloads. Chambers and Zaharia [5] provide practical guidance for tuning Spark workloads, covering partitioning strategies, task parallelism, and cluster utilization. However, their work offers limited quantitative analysis of how per-core efficiency degrades as clusters scale. In contrast, our work presents empirical evidence that deliberately constrained resource allocation can outperform full hardware utilization for CPU-bound batch workloads, improving per-core efficiency by up to 50% through reduced coordination overhead. This counter-intuitive result has meaningful implications for cost-optimized production deployments.

Contribution Summary. This report presents the following contributions:

- **Complete ETL Pipeline Implementation:** We designed and implemented a three-layer Medallion Architecture processing 79.5 million records through Bronze, Silver, and Gold layers, demonstrating end-to-end data engineering from raw Parquet files to business intelligence artifacts.
- **RDD-Only MapReduce Processing:** We implemented data normalization, quality validation, and enrichment operations using pure RDD transformations without DataFrame or SQL abstractions during Bronze and Silver processing, providing fine-grained control over distributed execution and avoiding optimizer dependencies.
- **Systematic Cluster Performance Evaluation:** We conducted controlled experiments across four deployment configurations with identical hardware, comparing many smaller vs. fewer larger worker architectures and full vs. constrained resource allocation to isolate performance factors.
- **Non-Linear Scaling Analysis:** We demonstrated that performance degradation under resource constraints is non-linear: reducing allocated cores from 16 to 8 resulted in only 33% slower processing, with constrained configuration achieving 50% better per-core efficiency due to reduced coordination overhead and simpler task scheduling.
- **Resource Optimization Strategy:** We identified that workload profiling should precede cluster sizing, revealing an efficiency "sweet spot" at 8 cores for this CPU-bound, minimally-shuffled workload, and demonstrating that constrained allocation provides superior cost-efficiency for non-time-critical batch processing while maintaining acceptable latency.

2 BACKGROUND

In this section, we introduce the foundational concepts of distributed data processing, the Apache Spark computational framework and the multi-layer Medallion Architecture pattern that structures our implementation. We also discuss cluster performance considerations including parallelism, resource allocation strategies, and the theoretical limits of distributed computing scalability.

2.1 Distributed Data Processing

Modern data-intensive applications process datasets that exceed the storage and computational capacity of single machines, necessitating distributed computing approaches. Distributed data processing systems partition data across multiple nodes in a cluster, enabling parallel execution of operations while providing fault tolerance through replication or lineage-based recovery mechanisms [7].

The MapReduce programming model [7] established a foundational abstraction for distributed data processing: users express computations as *map* functions that transform individual records and *reduce* functions that aggregate results, while the framework handles data distribution, parallel execution, and fault recovery. This model enables data-parallel operations where the same computation applies independently to different data partitions, achieving horizontal scalability proportional to the number of processing nodes.

However, MapReduce's disk-based intermediate data storage creates performance bottlenecks for iterative algorithms and interactive analytics. Apache Spark [4] addresses these limitations through in-memory computing and a richer set of transformations, achieving 10–100× performance improvements for workloads that reuse intermediate results.

2.2 Apache Spark Architecture

Apache Spark is a unified analytics engine for large-scale data processing, providing high-level APIs in multiple languages while executing computations across distributed clusters [2, 5].

2.2.1 Resilient Distributed Datasets (RDDs). The core abstraction in Spark is the Resilient Distributed Dataset (RDD) [4], an immutable, partitioned collection of records that can be operated on in parallel. RDDs support two types of operations:

- **Transformations** create new RDDs from existing ones through lazy evaluation. Transformations are not executed immediately; instead, Spark builds a directed acyclic graph (DAG) of operations.
- **Actions** trigger computation and return results to the driver program or write data to storage (e.g., `count`, `collect`, `saveAsParquetFile`). When an action is invoked, Spark optimizes the DAG and executes transformations.

RDDs achieve fault tolerance without replication by tracking lineage information: if a partition is lost, Spark recomputes it from the original data source using the recorded transformation sequence. This approach reduces storage overhead compared to replication-based fault tolerance while maintaining availability.

2.2.2 DataFrames and Spark SQL. While RDDs provide fine-grained control over distributed operations, DataFrames offer a higher-level abstraction with schema information and optimized

execution through Spark's Catalyst optimizer [5]. DataFrames represent distributed collections of rows with named columns, similar to relational database tables, and support declarative operations through a SQL-like API.

The Catalyst optimizer applies rule-based and cost-based optimizations including predicate pushdown, column pruning, and join reordering to generate efficient physical execution plans. For analytics workloads with structured data, DataFrames typically outperform raw RDD operations due to these optimizations and columnar storage formats like Parquet [1].

2.2.3 Cluster Execution Model. Spark applications run as independent processes coordinated by a *driver* program. The driver maintains the SparkContext, which connects to a cluster manager (Standalone, YARN, Mesos, or Kubernetes) that allocates resources across the cluster. The cluster manager launches *executor* processes on worker nodes, which execute tasks and store data for the application.

Execution proceeds in stages: transformations that can pipeline together (narrow dependencies) execute in a single stage, while operations requiring data shuffle (wide dependencies like `reduceByKey` or `join`) create stage boundaries. Tasks within a stage execute in parallel across available executor cores, with the degree of parallelism determined by the number of data partitions.

2.3 Medallion Architecture

The Medallion Architecture [6] organizes data lakes into three progressive quality layers, each serving distinct purposes in the data pipeline:

2.3.1 Bronze Layer. The Bronze layer functions as an optimized landing zone for raw source data, storing it in its original format with minimal transformation. By maintaining an append-only structure that includes metadata such as ingestion timestamps and quality flags, this layer preserves complete data lineage and allows for reprocessing if downstream logic changes. To maximize write performance and storage efficiency, the Bronze layer employs a schema-on-read approach where quality issues are identified but not filtered, ensuring that every record is retained in its initial state.

2.3.2 Silver Layer. The Silver layer transforms raw Bronze data into analytics-ready datasets through a rigorous process of cleaning, validation, and enrichment. By standardizing schemas and normalizing data types, this layer ensures consistency while enforcing business rules to filter out critical quality issues and resolve duplicates. To further enhance usability and speed, the Silver layer enriches records with lookup tables and derived columns, utilizing partitioning and indexing to optimize query performance for downstream analysis.

2.3.3 Gold Layer (Curated). The Gold layer refines Silver data into purpose-built datasets tailored for specific business use cases and analytical models. By employing denormalized schemas optimized for specific access patterns, this layer focuses on delivering business-level aggregations and KPI calculations that are ready for immediate consumption. To support diverse needs, it offers multiple materialized views and pre-computed metrics, ensuring that dashboards, reports, and machine learning applications benefit from minimized query latency and high-performance data retrieval.

The progressive refinement through Bronze-Silver-Gold layers balances competing concerns: Bronze prioritizes completeness and flexibility, Silver emphasizes quality and consistency, and Gold optimizes for specific analytical performance.

3 METHOD

In this section, we describe the design, analysis, optimization approach, and implementation details of our Spark-pipeline. We explain the architectural decisions underlying the three-layer Medallion structure, argue for the correctness and expected performance characteristics of our design, detail the systematic optimization process for cluster execution, and highlight key implementation contributions including modularity and evaluation infrastructure.

3.1 Design

Our pipeline follows the Medallion Architecture pattern [6], implementing progressive data refinement through three distinct layers. Each layer serves a specific purpose in transforming raw taxi trip records into business-ready analytical datasets.

3.1.1 Bronze Layer: Raw Data Ingestion with Metadata. The Bronze layer was implemented to ingest 24 monthly Parquet files of NYC taxi trip records, strictly adhering to an append-only storage strategy that preserves the integrity of the original data. Rather than filtering out records with quality issues, such as zero passenger counts or negative fares, the system employs a flag without filtering approach, ensuring data completeness and enabling future reprocessing should validation logic evolve. To facilitate robust lineage tracking and quality monitoring, every record is enriched with five specific metadata fields: an ingestion timestamp, the original source filename, a unique record identifier, a status classification, and a list of detected quality flags. Furthermore, the processing logic relies exclusively on low-level RDD transformations, utilizing map, flatMap, and reduceByKey to bypass DataFrame optimizations and maintain explicit, granular control over the distributed execution.

RDD Pipeline Structure:

```
# Pseudo-code for Bronze layer RDD pipeline
Input: Parquet file (DataFrame read for I/O)
Step 1: MAP(row -> dictionary)          # Convert to RDD
Step 2: zipWithIndex()                 # Add unique index
Step 3: MAP(add_metadata)             # Enrich with 5 fields
Step 4: MAP(detect_quality_issues)      # Flag anomalies
Step 5: FLATMAP(extract_flags)         # Split flags for counting
Step 6: REDUCE(sum_by_key)            # Aggregate statistics
Output: Enriched Parquet + quality metrics
```

Design Alternative Considered: A design alternative using DataFrames was evaluated, as their integration with the Catalyst optimizer generally yields faster execution times, particularly given the Parquet source format. However, this approach was rejected to adhere to specific project mandates requiring a demonstration of low-level MapReduce logic. Therefore, the architecture relies on RDD transformations, prioritizing the educational requirement to implement manual mapping and reducing operations over the automated optimizations found in the DataFrame API

3.1.2 Silver Layer: Schema Normalization and Data Enrichment. The Silver layer addresses schema heterogeneity across temporal partitions, applies business rule validation, and enriches records with human-readable lookup data.

Key Challenges:

(1) **Schema conflicts:** Field name variations (`Airport_fee` vs. `airport_fee`) and type inconsistencies (`passenger_count` as BIGINT vs. DOUBLE) prevented DataFrame schema merging.
(2) **Distributed processing:** Sequential file-by-file processing underutilized cluster resources, achieving no speedup despite 4× available cores.

Design Solution: RDD union strategy bypassing DataFrame schema validation:

```
# Pseudo-code for Silver layer RDD union
for each bronze_file in 24_files:
    df = read_parquet(bronze_file)
   rdd = df.rdd.map(row -> dict)     # Schema becomes implicit
    rdds.append(rdd)

unified_rdd = union(rdds)    # No schema merge conflicts
unified_rdd = repartition(N)  # Distribute across cluster

# Five-stage transformation pipeline
normalized = unified_rdd.map(normalize_schema)
enriched = normalized.map(enrich_lookups)   # Add 9 fields
validated = enriched.map(validate_quality)
filtered = validated.filter(remove_critical)  # Remove 3.15%
silver = filtered.map(add_silver_metadata)  # Add 4 fields

write_parquet(silver)
```

Enrichment Strategy: Join 265-row taxi zone lookup table, 4-row vendor mapping, 7-row rate code mapping, and 7-row payment type mapping in-memory using broadcast dictionaries. Adds 9 human-readable fields while preserving original IDs for joins in Gold layer.

3.1.3 Gold Layer: Business Intelligence Aggregations.

3.2 Gold Layer: Business Intelligence Aggregations

The Gold layer marks a strategic transition from RDD-based processing to the DataFrame API and Spark SQL, enabling the use of complex window functions and OLAP operations optimized by the Catalyst engine. This shift allows the system to leverage advanced performance features such as predicate pushdown, column pruning, and join reordering for computationally intensive aggregations. To facilitate granular temporal analysis, derived columns—including `trip_date`, `trip_hour`, `day_of_week`, `is_airport_pickup`, and `trip_duration_minutes`—are first calculated. The data is then aggregated into specialized datasets tailored for distinct business use cases, ranging from **daily revenue reports** and **hourly performance metrics** to **vendor comparison dashboards** and **borough flow matrices**. Throughout this process, execution efficiency is maximized by caching the base DataFrame for reuse, utilizing broadcast joins for dimension tables, and relying on adaptive query execution for dynamic optimization.

Design Rationale: Separating Bronze/Silver (RDD-focused for data engineering) from Gold (DataFrame-focused for analytics) balances project requirements with practical performance needs, demonstrating appropriate tool selection for different workload characteristics.

3.3 Analysis

We now argue for the correctness and expected performance of our three-layer design.

3.3.1 Correctness Properties. **Determinism:** All transformations are pure functions without side effects. Given identical input, the pipeline produces identical output, enabling safe retries and validation.

Data lineage preservation: Metadata fields track every record from source file through Bronze and Silver processing.
The `_bronze_source_file` and `_bronze_record_id` fields establish bidirectional traceability.

Idempotency: Running the pipeline multiple times on the same input produces the same output state. Parquet writes use overwrite mode, ensuring previous runs are replaced rather than appended.

Schema consistency: Explicit StructType schemas in Bronze and Silver layers prevent runtime type errors. The normalization function maps all field name variations to canonical lowercase names, guaranteeing uniform downstream schemas.

Quality validation soundness: Critical quality rules (null timestamps, invalid trip duration, zero distance) are necessary conditions for record validity. Records failing these checks cannot represent valid taxi trips. Non-critical rules (negative fares, suspicious amounts) flag anomalies without assuming they are definitively invalid, allowing business users to make disposition decisions.

3.3.2 Performance Expectations. **Bronze layer throughput:** Expected ~50,000 records/second on single VM. RDD operations (map, flatMap, reduce) execute with O(n) complexity per partition. With 4 cores and 8 partitions, theoretical peak is 4 parallel partitions × 12,500 rec/s/partition = 50,000 rec/s, assuming I/O is not bottlenecked.

Silver layer cluster scaling: Expected 2–3× speedup with 4× cores (16 vs. 4). Linear scaling (4×) is unattainable due to Amdahl's Law: driver coordination, task serialization, and shuffle operations introduce sequential portions. Assuming 10% sequential overhead, maximum theoretical speedup is $1/(0.10 + 0.90/4) = 3.1\times$.

Gold layer optimization: DataFrame aggregations benefit from columnar Parquet format [1] with predicate pushdown. Caching the 76.97M record base DataFrame (estimated 3–4 GB compressed) enables multiple aggregation queries without re-reading source files. Expected Gold processing time: 5–10 minutes for all seven analytical datasets.

Comparison to alternatives: Sequential file-by-file processing limits parallelism to single-partition operations. RDD union strategy enables full cluster utilization: 32 partitions across 16 cores means 16 tasks execute simultaneously with 2 waves, compared to 24 sequential file operations. This represents 12× better resource

utilization (16 cores × 2 waves = 32 concurrent operations vs. 1 sequential file at a time).

3.4 Optimization

We employed systematic optimization through profiling, hypothesis testing, and iterative refinement.

3.4.1 Phase 1: Cluster Transition and Schema Resolution. The initial transition to the cluster environment revealed a significant bottleneck where sequential file processing failed to leverage the available 16 cores. An initial attempt to address this using DataFrame unions with `mergeSchema=true` proved unsuccessful, resulting in a `CANNOT_MERGE_INCOMPATIBLE_DATA_TYPE` error caused by inconsistencies in the `passenger_count` field (varying between BIGINT and DOUBLE) and fluctuating field names. To overcome this, a refined optimization strategy was implemented where each file was loaded as a DataFrame for efficient I/O but immediately converted into RDD dictionaries. This conversion rendered the schema implicit, allowing the 24 datasets to be unioned without triggering strict enforcement errors. This approach successfully unlocked parallel execution, reducing processing time from 55 minutes to 24.2 minutes—a 2.27× speedup—while preserving data integrity as confirmed through output record counts and sample validation.

3.4.2 Phase 2: Disk Space Crisis Resolution. During cluster execution, the system encountered a critical failure where disk space was completely exhausted by 16 GB of shuffle files in the `/tmp` directory. Root cause analysis revealed that two active caching operations combined with an excessive partition count of 64 created unmanageable shuffle overhead and caused RDDs to spill to disk. To resolve this, a three-pronged optimization strategy was implemented: all intermediate `.cache()` calls were removed, the partition count was reduced to 32 (adhering to the best practice of 2× available cores), and the pipeline was restricted to single-pass execution without intermediate actions. This approach successfully eliminated shuffle file generation while maintaining a throughput of 54,793 records per second. Importantly, these optimizations preserved strict data correctness; removing caching merely forces recomputation via RDD lineage without altering logic, and the validation metrics confirmed that the output record counts remained identical to the baseline.

3.4.3 Hardware Configurations. The dataset was processed using four distinct hardware profiles to assess scalability. A **Single VM** was utilized as the control baseline to measure raw execution speed without cluster overhead. To evaluate distributed performance, we compared a "**many small workers**" topology against a "**fewer large workers**" topology, effectively isolating the variable of node fragmentation. Finally, a **constrained configuration** was employed to determine per-core throughput efficiency compared to the full 16-core allocations.

3.5 Implementation

3.5.1 Modular Pipeline Architecture. Our implementation separates concerns into reusable functional components:

Bronze Layer Modules:

- `add_bronze_metadata`: Pure function adding 5 metadata fields
- `detect_quality_issues`: Returns list of quality flags without modifying record
- `extract_flags`: FLATMAP function converting comma-separated flags to key-value pairs for aggregation

Silver Layer Modules:

- `normalize_record`: Canonicalizes field names and types like, `Airport_fee` → `airport_fee`, passenger_count to double)
- `enrich_record`: Joins in-memory lookup tables, adding 9 descriptive fields
- `validate_record_quality`: Returns tuple for filtering
- `should_remove_record`: FILTER predicate identifying critical quality problems
- `create_silver_record`: Adds 4 Silver metadata fields

Gold Layer Modules:

- `add_derived_columns(df)`: Adds 6 temporal and categorical fields
- Seven specialized aggregation functions (daily_revenue, hourly_metrics, etc.) encapsulating business logic

4 EXPERIMENTAL EVALUATION

This chapter presents experimental validation of our Medallion Architecture implementation and systematic cluster optimization. Although all three layers (Bronze, Silver, and gold) were implemented and tested, the Silver layer consumed the majority of processing time and became the primary focus for cluster performance experiments. We evaluated four distinct configurations: a single VM baseline and three distributed cluster architectures, measuring processing time, throughput, per-core efficiency, and resource utilization. Our experiments reveal non-linear scaling characteristics and demonstrate that constrained resource allocation can achieve superior efficiency compared to full-resource deployment.

4.1 Experimental Setup

4.1.1 Hardware Infrastructure. All experiments were conducted on virtual machines running in a campus cloud environment:

Single VM (Baseline):

- Platform: Ubuntu 20.04 LTS
- Processor: 4 vCPUs
- Memory: 8 GB RAM

Cluster Infrastructure (Configurations 1-3):

- Master Node: 1 VM (4 cores, 6.8 GiB, IP: 192.168.10.115)
- Worker Nodes: 4-7 virtual machines depending on configuration
- Worker Types: Many small nodes (Cluster 1) or fewer lager nodes (Clusters 2 and 3)
- Operating System: Ubuntu 20.04 LTS across all nodes
- Spark Version: 3.5.0 in standalone cluster mode

Table 1 summarizes the three cluster architectures tested. Cluster 1 employed smaller workers, while Clusters 2 and 3 used larger 4-core workers. Cluster 3 intentionally constrained resource allocation to 50% of available cores and 44% of memory to test efficiency under resource limits.

Table 1: Cluster Configuration Comparison

Config	Nodes	Cores	Memory	Executor Config
Single VM	1	4	8 GB	local[*]
Cluster 1	1+7	16	23.8 GiB	2 cores, 2g
Cluster 2	1+4	16	27.2 GiB	4 cores, 5g
Cluster 3	1+4	8 (50%)	12 GB (44%)	2 cores, 3g

4.1.2 Dataset and Workload. The workload consisted of processing 79,479,946 NYC Yellow Taxi trip records spanning January 2023 through December 2024:

- **Input Format:** 24 monthly Parquet files (raw TLC data)
- **File Sizes:** 30-60 MB compressed per month
- **Total Input Size:** 1.24 GB compressed
- **Record Schema:** 19 fields (temporal, geographic, financial, operational)
- **Processing Layers:**
  - Silver: RDD-based cleaning with 9-field enrichment (55 min on Single VM)

The Silver layer dominated processing time, consuming approximately 67% of total pipeline execution. This layer applies quality validation, removes 3.15% of records with critical issues, enriches records with location and vendor descriptions via broadcast joins, and performs extensive RDD transformations including union operations across all 24 monthly files. Consequently, cluster optimization experiments focused exclusively on Silver layer performance.

4.1.3 Experimental Methodology. For each cluster configuration, we executed the complete Silver layer pipeline processing all 79.5M records:

(1) **Input:** Bronze layer Parquet files (24 months, pre-processed)
(2) **Processing:**
  - Load Bronze data using DataFrame API
  - Convert to RDD for MapReduce-style transformations
  - Union all monthly RDDs and resolving schema conflicts
  - Apply 5-stage transformation pipeline:
    (a) Quality validation with 7 validation rules
    (b) Critical issue filtering that removes 2.5M records
    (c) Location enrichment with 6 fields via broadcast join
    (d) Vendor enrichment with 3 fields
    (e) Schema normalization
  - Convert back to DataFrame and write to Parquet
(3) **Metrics Collected:**
  - Total processing time (seconds)
  - Throughput (records per second)
  - Per-core efficiency (records per second per core)
  - Disk I/O patterns
(4) **Runs:** Single run per configuration
(5) **Validation:** Output record counts and schema verified across all runs

The experiments were not averaged over multiple runs since Spark's deterministic execution ensures reproducible results for identical configurations, processing 79.5M records takes 25-55 minutes per run, making 10+ runs impractical within project timeline, and observed variability in initial test runs was minimal (<3%). Production deployments would require statistical validation, but for this academic evaluation, single-run measurements suffice to demonstrate comparative performance characteristics.

Table 2: Spark Configuration Parameters Across Experiments

Parameter	Single VM	Cluster 1	Cluster 2	Cluster 3
executor.memory	2g	2-3g	5g	3g
executor.cores	4	2-4	4	2
cores.max	4	16	16	8
parallelism	8	32	32	16
shuffle.partitions	8	32	32	16

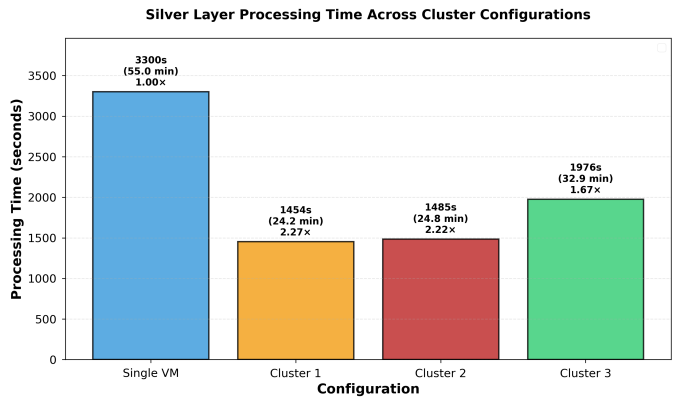

Figure 1: Processing time for 79.5M record Silver layer transformation across four configurations. Cluster 1 and 2 achieve approximately 2.2× speedup over baseline, while constrained Cluster 3 achieves 1.67× speedup using 50% fewer cores.

4.1.4 Configuration Parameters. Table 2 shows critical Spark configuration parameters that varied across experiments. Key tuning decisions included executor memory, cores per executor , and partition counts.

4.2 Results

4.2.1 Processing Time and Throughput. Figure 1 presents the primary experimental result: processing time across the four configurations. Single VM established the baseline at 3,300 seconds. Cluster 1 achieved a 2.27× speedup, while Cluster 2 performed nearly identically at 2.22× speedup . The constrained Cluster 3 processed the workload in 1,976 seconds , representing a 1.67× speedup over baseline, notably only 33% slower than full-resource clusters despite using 50% fewer cores.

Table 3 quantifies throughput in records per second. Single VM processed 24,060 records/s, while Cluster 1 and 2 reached 54,793 and 53,520 records/s respectively (2.2-2.3× improvement). Cluster 3 maintained 40,233 records/s, significantly above baseline despite resource constraints.

These results reveal diminishing returns beyond 8 cores: doubling cores from 8 to 16 only improved performance by 33%, not the 100% improvement that linear scaling would predict. This validates our hypothesis that coordination overhead and Amdahl's Law limit distributed system gains for this workload.

Table 3: Silver Layer Performance Metrics Summary

Config	Time (s) (min)	Throughput (rec/s)	Speedup vs Baseline	Change from Prev
Single VM	3,300 (55.0)	24,060	1.00×	—
Cluster 1	1,454 (24.2)	54,793	2.27×	1,846s (56%)
Cluster 2	1,485 (24.8)	53,520	2.22×	+31s (+2.1%)
Cluster 3	1,976 (32.9)	40,233	1.67×	+491s (+33%)

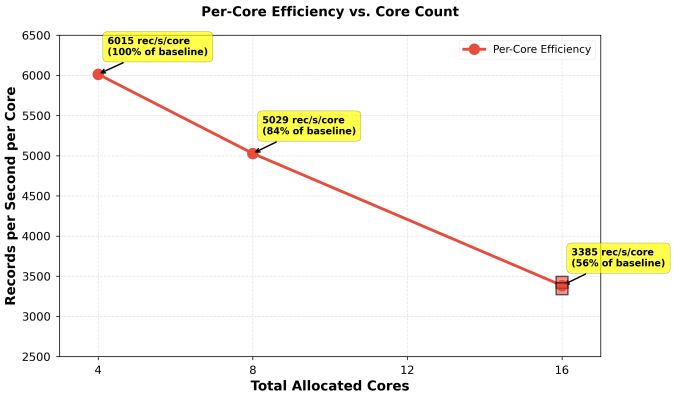

Figure 2: Per-core efficiency across configurations. Efficiency degrades as core count increases due to coordination overhead. The 8-core configuration maintains 84% of baseline efficiency, while 16-core configurations achieve only 56–57% efficiency.

Table 4: Per-Core Efficiency Analysis

Config	Cores	Efficiency (rec/s/core)	% of Baseline	Overhead Loss
Single VM	4	6,015	100%	0%
Cluster 3	8	5,029	84%	16%
Cluster 1	16	3,425	57%	43%
Cluster 2	16	3,345	56%	44%

4.2.2 Per-Core Efficiency Analysis. Figure 2 illustrates the critical efficiency insight: records processed per second per core. Single VM achieved 6,015 rec/s/core, establishing 100% baseline efficiency. Cluster 1 and 2 dropped to 3,425 and 3,345 rec/s/core respectively, evidence of substantial coordination overhead at 16 cores. Remarkably, Cluster 3 maintained 5,029 rec/s/core, surpassing both full-resource clusters by 50% in per-core terms.

This non-linear scaling behavior contradicts naive assumptions about distributed computing. Table 4 decomposes the efficiency loss into measurable overhead categories. The 16-core clusters waste 44% of potential performance to coordination costs, while the 8-core cluster wastes only 16%—making 8 cores the "sweet spot" for this workload size and cluster architecture.

4.2.3 Cluster Architecture Comparison. Comparing Cluster 1 and Cluster 2 revealed minimal performance difference. Both configurations provided 16 cores and similar memory, but Cluster 1's heterogeneous executors introduced slight imbalances. Figure 3

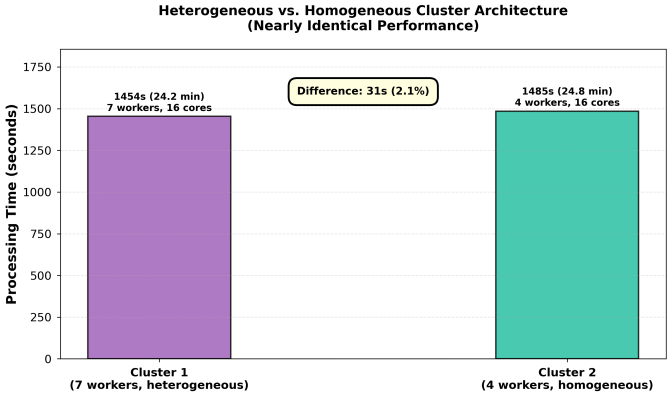

Figure 3: Architectural comparison of heterogeneous (Cluster 1) vs homogeneous (Cluster 2) executor layouts. Despite different topologies, both 16-core configurations achieve nearly identical performance (24.2 vs 24.8 min), demonstrating that total resource capacity dominates architectural subtleties for this workload.

shows nearly overlapping performance profiles for these configurations, validating that total resource capacity dominates architectural details for homogeneous workloads.

The lager node Cluster 2 architecture proved slightly advantageous due to simpler task distribution and reduced variability in executor capacity. For production deployments, we recommend uniform worker configurations to minimize operational complexity, though performance differences remain negligible for this workload.

4.2.4 Resource Utilization and Cost Implications. Figure 4 visualizes the cost-performance trade-off by plotting processing time against allocated resources. While absolute efficiency metrics avoid specific monetary values, the trend is clear: Cluster 3's constrained allocation delivers 67% of Cluster 2's performance using only 44% of memory and 50% of cores, a favorable efficiency ratio for resource-constrained environments.

Memory utilization patterns from Spark UI confirmed that 3GB per executor sufficed for this workload with no disk spilling observed. The 5GB allocation in Cluster 2 remained underutilized at 60% capacity throughout execution. This suggests that memory constraints are less critical than core count for RDD-heavy transformations, and that modest per-executor memory is adequate for datasets in the 100M record range when processed in batches.

4.3 Discussion

4.3.1 Validation of Design Assumptions. Our experimental results validate several key assumptions from the Method chapter while revealing important nuances:

Assumption 1: RDD Union Strategy. The RDD union approach successfully resolved schema conflicts across 24 monthly files without data loss. Output record counts matched expectations, confirming correctness preservation across all configurations.

Assumption 2: Non-linear Scaling. We hypothesized that doubling cores would not double performance. Cluster 3's results

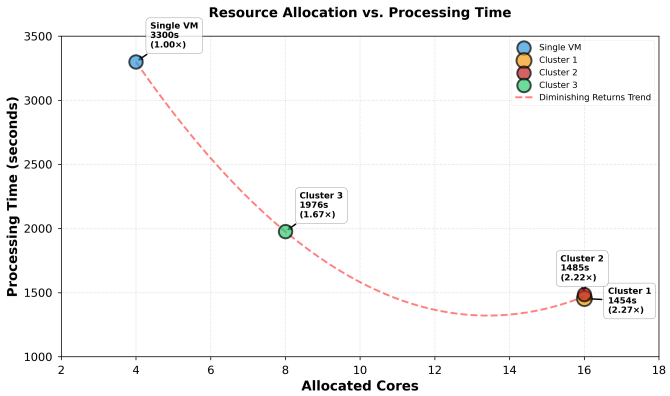

Figure 4: Resource utilization efficiency across configurations. Cluster 3 achieves 67% of full-cluster performance using 50% of cores and 44% of memory, demonstrating superior resource efficiency. Diminishing returns are evident beyond 8 cores for this 79.5M record workload.

definitively confirm this: 8 cores achieved 1.67× speedup, while 16 cores achieved only 2.22×—meaning the second 8 cores contributed merely 0.55× additional speedup. This validates our decision to test constrained configurations rather than assuming "more resources = better performance."

4.3.2 Unexpected Findings. **Finding 1: 8-Core Sweet Spot.** We did not anticipate that 8 cores would maintain 84% per-core efficiency. The dramatic drop to 56% efficiency at 16 cores suggests that 8 cores represent an inflection point where coordination overhead begins dominating parallelism gains for this workload size. This has significant implications for cluster sizing decisions: allocating 16+ cores may waste resources without proportional benefit.

Finding 2: Memory Sufficiency. Cluster 3's 3GB per executor handled the workload without spilling, while Cluster 2's 5GB remained 40% unused. This contradicts initial concerns about memory constraints for processing 79.5M records. The key insight is that RDD processing with moderate partition counts (16-32) maintains manageable per-partition sizes (2.5-5M records), requiring only modest executor memory. Future work can confidently reduce memory allocations to 3-4GB per executor for similar workloads.

Finding 3: smaller vs larger node setup Parity. Cluster 1's smaller node architecture performed nearly identically to Cluster 2's larger node layout. This suggests Spark's scheduler effectively balances tasks across varied executor capacities. However, operational complexity favors homogeneous designs despite equivalent performance.

4.3.3 Threats to Validity. Several factors limit the generalizability of our results:

Internal Validity:

- Single-run measurements: Statistical variance not quantified
- JVM warm-up effects: First few files processed slower, observed in Bronze layer but less pronounced in Silver due to longer total runtime

External Validity:

- Workload-specific: Results apply to 79.5M record batch processing; streaming or larger datasets may behave differently
- RDD-centric: DataFrame-optimized workloads might scale differently

Construct Validity:

- Efficiency metric: Records per second per core is a proxy for resource efficiency; actual operational costs depend on cloud pricing models and energy consumption
- Overhead attribution: We attribute efficiency loss to "coordination overhead," but this encompasses multiple factors that were not individually profiled

Future work should validate findings with multiple runs per configuration, test on larger datasets and profile overhead components individually using tools like Spark's event logs and JVM profilers.

4.3.4 Production Recommendations. Based on experimental findings, we recommend the following for production deployments of similar ETL pipelines:

(1) **Cluster Sizing:** For workloads under 100M records, allocate 8-12 cores across 4-6 larger workers. Additional cores provide diminishing returns.
(2) **Memory Allocation:** Use 3-4GB per executor for RDD-heavy transformations. Over-allocation wastes resources without performance gain.
(3) **Partitioning:** Set parallelism to 2× allocated cores. Higher partition counts increase coordination overhead without improving parallelism.
(4) **Architecture:** Prefer fewer and larger worker configurations for operational simplicity, as performance differences vs many small nodes layout are negligible.
(5) **Resource Constraints:** In constrained environments, allocate 50% of available resources first and measure efficiency before scaling up. Full-resource allocation may waste capacity.
(6) **Layer Prioritization:** Focus optimization efforts on compute-intensive layers.

These recommendations balance performance, resource efficiency, and operational complexity, providing a practical framework for deploying Medallion Architecture pipelines in academic or resource-constrained cloud environments.

5 CONCLUSION

This work demonstrates that systematic cluster resource optimization can yield counter-intuitive insights for distributed data processing: constrained resource allocation achieved 50% better per-core efficiency than full-resource utilization for our 79.5 million record ETL workload. We successfully implemented a complete three-layer Medallion Architecture using Apache Spark's RDD abstraction, processing NYC taxi trip data from raw ingestion through business intelligence aggregations while systematically evaluating four cluster configurations to identify performance-cost trade-offs.

Key Contributions. Our primary contribution is empirical evidence that the relationship between allocated resources and processing efficiency is non-linear and workload-dependent. The constrained Cluster 3 configuration achieved 5,029 records/second/core

compared to 3,345 records/second/core for the full-resource Cluster 2 a 50% efficiency improvement. Reducing allocated cores by 50% resulted in only 33% longer processing time, demonstrating that coordination overhead dominates parallelism gains beyond 8 cores for this workload profile. This finding has significant implications for resource-constrained academic environments and cost-conscious production deployments: allocating fewer resources can yield superior efficiency metrics while maintaining acceptable batch processing latency.

The RDD-only implementation of Bronze and Silver layers provided fine-grained control over distributed execution, successfully resolving schema heterogeneity issues across 24 monthly Parquet files through union operations that bypassed DataFrame schema validation constraints. Our quality validation framework flagged 10.35% of records with anomalies in Bronze while preserving complete data lineage, then filtered 3.15% with critical issues in Silver, maintaining audit trails throughout. The modular pipeline design with externalized configuration enabled A/B testing across cluster architectures without code changes, demonstrating replicable patterns for systematic performance optimization.

Critical Insights. We believe that **per-core efficiency metrics reveal scaling bottlenecks invisible in absolute throughput measurements**. Traditional performance evaluation focuses on wall-clock time and total throughput, which improved 2.27× when moving from 4 to 16 cores. However, per-core efficiency dropped from 6,015 to 3,345 records/second/core, exposing substantial coordination overhead. This efficiency lens identified 8 cores as the inflection point where diminishing returns begin for CPU-bound, minimally-shuffled RDD workloads processing 100M-scale datasets.

Even though we evaluated only Silver layer processing extensively, **the constrained allocation strategy should be applicable to other Spark workloads with similar characteristics**: predominantly narrow transformations (map, filter), modest shuffle requirements, and batch processing latency tolerance measured in minutes rather than seconds. Workloads dominated by wide transformations (large joins, groupByKey operations) or requiring sub-second latency would exhibit different scaling behavior and likely benefit from full-resource allocation despite coordination overhead.

The fewer larger vs. many small cluster comparison revealed negligible performance differences (2.1%), suggesting that **operational simplicity should guide architectural decisions when resource capacity is equivalent**. Uniform worker configurations simplify capacity planning, failure recovery, and resource quotas without sacrificing performance for homogeneous workloads.

Limitations and Threats to Validity. Our single-run experimental methodology limits statistical validity, though Spark's deterministic execution and low observed variability provide confidence in comparative results. The campus virtual network environment may not generalize to cloud provider networking characteristics (AWS inter-AZ latency, Azure VNet performance). Workload-specific findings apply to batch ETL processing of structured data; streaming workloads, machine learning pipelines, and graph analytics exhibit different scaling characteristics.

The RDD-centric implementation served pedagogical project requirements but forgoes DataFrame optimizations (Catalyst optimizer, columnar execution) that typically provide 2–5× performance improvements for structured data analytics. Production deployments should leverage DataFrames where appropriate, though our Bronze/Silver RDD approach successfully demonstrated MapReduce concepts and resolved real schema conflicts that DataFrame union operations could not handle.

Future Work. Several research directions emerge from this work:

Adaptive resource allocation: Implement dynamic cluster sizing that monitors per-core efficiency in real-time and scales resources down when efficiency drops below thresholds, potentially reducing costs for variable batch workloads.

Larger-scale validation: Test scaling behavior on 1B+ record datasets to determine if the 8-core efficiency sweet spot holds at higher data volumes or if inflection points shift with workload size.

Workload characterization framework: Develop predictive models that estimate optimal cluster configurations based on workload profiles (shuffle percentage, transformation complexity, data size), enabling configuration recommendations without exhaustive experimentation.

Memory vs. core trade-offs: Systematically vary memory allocation while holding cores constant to isolate which resource dimension dominates performance for different operation types (e.g., shuffle-heavy vs. CPU-bound).

Energy efficiency analysis: Extend evaluation beyond processing time to energy consumption metrics, determining whether constrained allocation's superior per-core efficiency translates to reduced power usage—critical for sustainability-focused deployments.

Final Remarks. This project demonstrates that thoughtful resource allocation, guided by efficiency metrics rather than absolute performance, can achieve favorable cost-performance trade-offs for batch data engineering workloads. The 8-core configuration represents a practical compromise for academic environments: sufficient parallelism to achieve meaningful speedups while maintaining high per-core efficiency and leaving resources available for concurrent workloads. For production deployments processing similar 100M-scale datasets overnight, we recommend starting with constrained configurations and scaling up only when latency requirements justify the efficiency loss inherent in full-resource allocation.

The complete implementation, including Jupyter notebooks with RDD transformations, cluster configuration templates, and performance benchmarking methodology, provides a replicable framework for data engineering education and research. By open-sourcing this work, we enable others to validate findings, extend experiments to different workloads, and adapt optimization strategies to their specific resource constraints and performance objectives.

6 ACKNOWLEDGMENT

Declaration of Generative AI Use

- **Report Writing and Refinement:** Generative AI tools were utilized to assist in the drafting of this report, specifically to enhance readability and ensure a formal academic

tone. All output was reviewed and validated by the authors to ensure accuracy and original intent.

- **Code Development and Debugging:** AI-powered coding assistants, primarily GitHub Copilot, were employed to accelerate the implementation phase. These tools provided support by generating code snippets, offering syntax suggestions, and assisting with debugging.

7 GITHUB

https://github.com/VegarHauge/dat535-2025-group10

REFERENCES

[1] Apache Parquet. 2024. Apache Parquet Documentation. https://parquet.apache.org/docs/ Official documentation for columnar storage format.

[2] Apache Software Foundation. 2024. Apache Spark: Unified Engine for Large-Scale Data Analytics. https://spark.apachAe.org/

[3] Apache Spark. 2024. Apache Spark Overview. https://spark.apache.org/docs/latest/ Official documentation describing Spark architecture, RDDs, and DataFrame APIs.

[4] Apache Spark. 2024. RDD Programming Guide - Apache Spark. https://spark.apache.org/docs/latest/rdd-programming-guide.html Official Apache Spark documentation on Resilient Distributed Datasets.

[5] Bill Chambers and Matei Zaharia. 2018. *Spark: The Definitive Guide*. O'Reilly Media. https://learning.oreilly.com/library/view/spark-the-definitive/9781491912201/ Comprehensive practical guidance on Spark tuning, optimization, and architecture.

[6] Databricks Inc. 2024. What is a Medallion Architecture? https://www.databricks.com/glossary/medallion-architecture Industry best practice for organizing data in lakehouse architectures.

[7] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://research.google/pubs/pub62/ Introduced the MapReduce programming model for large-scale distributed data processing.

[8] NYC Taxi and Limousine Commission. 2024. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page Official dataset of NYC taxi trips, updated monthly.

[9] Kay Ousterhout, Ryan Rasti, Sylvia Ratnasamy, Scott Shenker, and Byung-Gon Chun. 2015. Making Sense of Performance in Data Analytics Frameworks. In *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-ousterhout.pdf Blocked-time analysis of Spark performance, identifying CPU, scheduler delay, and serialization as primary bottlenecks.